



Guida sullo svolgimento di esercizi di Prolog del- l'esame di Linguaggi di Programmazione 1

GIUGNO 2025

Paolo Liberti
N86004255

Indice

1	Introduzione	3
1.1	Come utilizzare il documento	3
2	Tipologie di esercizi	4
2.1	Riempimento di una lista vuota	4
2.2	Scorrimento di una lista	5
2.3	Riempimento inverso di una lista e MGU	6
2.4	Utilizzo di un contatore	7
2.5	Rimuovere (o saltare) un elemento	9

Capitolo 1

Introduzione

1.1 Come utilizzare il documento

Questa è una guida sviluppata per l'esame di Linguaggi di Programmazione su come affrontare al meglio gli esercizi di Prolog da svolgere durante l'esame scritto.

Ovviamente questa non è una guida ufficiale che sostituisce lo studio attraverso le slide del professore e i libri, bensì un modo per facilitare la comprensione degli esercizi e delle tipologie che compaiono.

E' molto importante integrare con altri esempi e uno studio rigoroso, Prolog è un linguaggio funzionale che richiede un ragionamento da utilizzare ben diverso da linguaggi comuni come C, Java o C++, per questo motivo ogni tipologia d'esercizio presente avrà una spiegazione a livello logico per permettere la comprensione di tali.

Nel documento sono presenti alcune tipologie di esercizi svolti in esami passati, ovviamente possono uscire esercizi ben diversi da questi, ma tali dovrebbero essere una buona base per capire come sviluppare un ragionamento in grado di risolvere ogni problema di Prolog.

Detto questo, passiamo alle varie tipologie di esercizi.

Capitolo 2

Tipologie di esercizi

2.1 Riempimento di una lista vuota

Questa tipologia è riconoscibile dal fatto che nella query è presente una lista piena e una (o più) variabile logica già definita da dover riempire come se fosse una lista, ricordiamo che le variabili logiche non hanno un tipo specifico, quindi è assolutamente possibile trattarle come tali.

```
//Implementare in Prolog il predicato split(L, Val, L1, L2) che divide
    la lista L in due parti: in L1 gli elementi numerici pi piccoli di
    Val e in L2 quelli pi grandi. Se L contiene anche valori non
    numerici, questi vengono ignorati. Esempio di utilizzo:
?- split ([a, 8, 3, d, 256, 10, ciao], 10, A, B).
A = [8, 3],
B = [256].
```

Questo è l'esercizio di Giugno 2025, un esercizio dove riempire due liste già definite ma vuote attraverso una serie di limitazioni.

Questo codice risolve l'esercizio:

```
split([H|T], X, [H|L1], L2):- number(H), H<X, split(T, X, L1, L2).
split([H|T], X, L1, [H|L2]):- number(H), H>X, split(T, X, L1, L2).
split([H|T], X, L1, L2):- not(number(H)), split(T, X, L1, L2).
split([H|T], X, L1, L2):- X==H, split(T, X, L1, L2).
split([], _, [], []).
```

Passiamo quindi alla spiegazione:

Prima di tutto, analizzando l'esercizio possiamo comprendere 3 condizioni:

- Se il numero è più piccolo del secondo elemento, esso dovrà essere inserito alla testa di L1
- Se invece è più grande, dovrà essere inserito alla testa di L2
- Se non è un numero OPPURE è lo stesso valore del secondo elemento (anche se non specificato, notate come nell'esempio il numero "10" viene ignorato!).

```
split([H|T], X, [H|L1], L2):- number(H), H<X, split(T, X, L1, L2).
```

Nella prima riga, iniziamo a preparare il riempimento della lista in testa, possibile attraverso la parte definita in `[H—L1]` nella head (questo è possibile perchè nella query è presente una variabile logica vuota). Nel body invece prepariamo le condizioni per il riempimento di `L1`, ovvero assicurarci che `H` è un numero (con il predicato built-in "number") e che sia più piccolo di `X`. Una volta fatto ciò eseguiamo la chiamata ricorsiva con gli argomenti presenti nel codice qui sopra.

```
split([H|T], X, L1, [H|L2]):- number(H), H>X, split(T, X, L1, L2).
```

Nella seconda riga eseguiamo le medesime operazioni, con la differenza di applicare le condizioni per la lista `L2` citate in precedenza.

```
split([H|T], X, L1, L2):- not(number(H)), split(T, X, L1, L2).
```

Stavolta trattiamo la condizione per cui NON dobbiamo riempire le liste (ricordatevi sempre le condizioni non favorevoli all'esercizio, è la base per ogni programmatore!), prima di tutto controlliamo il caso in cui `H` non è un numero attraverso "not", che inverte il risultato di matching di un predicato.

```
split([H|T], X, L1, L2):- X==H, split(T, X, L1, L2).
```

Nella quarta riga specifichiamo il caso in cui `H` e `X` corrispondono, utilizzando l'operatore "==" di equivalenza.

```
split([], _, [], []).
```

Da qui creiamo il nostro caso finale, riconoscibile perchè la lista originale è stata visitata nella sua interezza (primo elemento `[]`), a differenza di altri esercizi, non servono particolari casi di matching, quindi poniamo le nostre variabili a `[]`.

2.2 Scorrimento di una lista

Questa è una delle tipologie più semplici, il nostro obiettivo è quello di scorrere una lista già definita, restituendo true o false a seconda delle condizioni.

Implementare in Prolog il predicato `allOdds(L, X)` che vero quando `X` occupa tutte le posizioni dispari della lista `L` (contando le posizioni a partire da 1).

Ad esempio, `allOdds([a, b, a, d], a)` vero, mentre `allOdds([a, a, c, a], a)` falso.

Questo esercizio è comparso nell'esame di Settembre del 2023. Ecco la risoluzione:

```
allOdds([X], X).
allOdds([], _).
allOdds([X,Y|T], X):- X\=Y, allOdds(T, X).
```

Questo esercizio è svolgibile rapidamente con un semplice esercizio di ragionamento algebrico, svolgendo l'esercizio al contrario.

```
allOdds([X,Y|T], X):- X\=Y, allOdds(T, X).
```

Non a caso, la logica dietro questa riga è quella di assicurarsi che X non sia in posizioni pari, controllando che ogni seconda posizione nella lista nelle chiamate ricorsive abbia un valore diverso da X.

```
allOdds([X], X).
allOdds([], _).
```

I casi finali in questo caso sono due, poichè dobbiamo trattare i casi in cui la lista sia di lunghezza pari o dispari (questo perchè ad ogni chiamata visitiamo gli elementi a due a due, rendendo necessaria tale gestione).

Il primo caso in particolare si occupa di gestire il caso di lunghezza dispari, dovrebbe ritornare true, quindi lo gestiamo come un match.

2.3 Riempimento inverso di una lista e MGU

Stavolta analizziamo l'utilizzo di alcune tecniche, quali l'overloading e il MGU (Most General Unifier) nell'ultimo passo.

Implementare in Prolog il predicato `reverse` che inverte l'ordine degli elementi in una lista.

Esempi di utilizzo:

```
?- reverse([], []).
true.
?- reverse([a, b, c], [c, b, a]).
true.
?- reverse([1, 2, 3, 4, 5], R).
R = [5, 4, 3, 2, 1].
```

Questo esercizio è comparso nell'esame di Gennaio 2025, tende a essere proposto più volte, anche se indirettamente, ecco uno svolgimento:

```
reverse([], []).
reverse(L1, A):- reverse(L1, A, _).
reverse([H|T], A, B):- reverse(T, A, [H|B]).
reverse([], A, A).
```

In questo caso vengono utilizzate varie tecniche non mostrate in precedenza, come sempre analizziamo ogni riga per capire cosa facciamo:

```
reverse([], []).
```

Nella prima riga, assicuriamoci di poter avere un true se le liste sono vuote, come accade nel primo esempio.

```
reverse(L1, A):- reverse(L1, A, _).
```

La logica in esercizi come questo è quello di agire su una nuova lista creata da noi, per poi fare una unificazione con il secondo elemento (che può essere libero) nella parte finale. Per fare ciò effettuiamo un overloading su reverse, per aggiungere un terzo elemento.

```
reverse([H|T], A, B):- reverse(T, A, [H|B]).
```

Da qui, ora dobbiamo riempire la nostra nuova variabile in maniera inversa rispetto alla prima lista, ragionandoci sopra un approccio basato su LIFO (Last In First Out) come avviene con lo stack potrebbe funzionare, ovvero:

```
[1]
[2,1]
[3,2,1]
[4,3,2,1]
[5,4,3,2,1]
```

Per fare ciò sfruttiamo la natura ricorsiva di Prolog, riempiendo la nuova lista con l'head della prima ad ogni chiamata.

```
reverse([], A, A).
```

A questo punto facciamo un'unificazione sfruttando l'algoritmo di MGU (Most General Unifier), in cui diciamo a prolog che il secondo elemento deve essere unificato con la nostra nuova lista. Ciò è semplice, poichè basta far combaciare il nome delle variabili.

Come potete vedere, un esercizio all'apparenza semplice può nascondere dei ragionamenti particolari nonostante il codice lessicalmente corto. E' quindi fondamentale ragionarci prima sopra, sviluppando un piano di svolgimento logico passo passo.

2.4 Utilizzo di un contatore

In molti esercizi è necessario creare e utilizzare una nuova variabile da utilizzare come un "contatore", questi esercizi sono riconoscibili perchè bisogna fare spesso confronti numerici o sommare più valori, per esempio assicurarci che X sia il massimo, una somma e così via...

In questo caso, valutiamo l'esercizio di Settembre 2024:

Implementare in Prolog il predicato `max(L, Sum)` che vero se `L` una lista che contiene almeno un valore numerico, e `Sum` la somma dei valori numerici contenuti. Ad esempio, `max([a, b, 20, d, 10], 30)` vero, mentre `max([a, b, d], S)` falso.
Suggerimento: il predicato built-in `number(X)` verifica se `X` un valore numerico.

A primo impatto, può sembrare un esercizio piuttosto complesso, ma con dei ragionamenti da fare prima, diventa molto semplice, ecco una soluzione:

```
max(L1, X):- max(L1, X, 0).  
max([H|T], X, Y):- number(H), Z is H+Y, max(T, X, Z).  
max([H|T], X, Y):- not(number(H)), max(T, X, Y).  
max([], X, X).
```

Prima di svolgere l'esercizio, è meglio spiegare cosa intendo per "contatore". Ciò non è altro che una variabile che solitamente inizializziamo con un valore numerico (in questo caso 0) per poter fare delle operazioni su di essa nelle successive chiamate, senza preoccuparci che abbia un valore vuoto.
In questo esercizio, ogni volta che dobbiamo sommare un valore numerico, non facciamo altro che salvarlo nel nostro contatore, riducendo questo esercizio a un banale processo matematico di facile risoluzione.
Ora passiamo alla spiegazione dell'esercizio:

```
max(L1, X):- max(L1, X, 0).
```

Come prima, effettuiamo un overloading per avere una nuova variabile, in questo caso inizializzata a 0.

```
max([H|T], X, Y):- number(H), Z is H+Y, max(T, X, Z).
```

Nel caso in cui `H` è un numero (verificato con il predicato built-in "number"), poniamo le condizioni per modificare il nostro contatore.
La logica è quella di aggiungere il valore della testa al nostro contatore, questo è possibile attraverso "`Z is H+Y`".
`Z` è una nuova variabile temporanea che utilizziamo per salvare la somma della testa in quel momento e del nostro contatore attraverso la parola "is", fondamentale per salvare operazioni aritmetiche in una variabile (ricordatevi che "=" in Prolog è ben diverso da C o Java, in quanto qui funge da unificatore, sulle slide del professore è presente una spiegazione più approfondita, consiglio altamente di vederla).

```
max([H|T], X, Y):- not(number(H)), max(T, X, Y).
```

In questo caso gestisco la situazione in cui non sia un numero, usando la stessa tecnica del primo esercizio.

```
max([], X, X).
```

Infine, sfrutto il MGU per valutare che il nostro contatore sia uguale alla somma richiesta.

2.5 Rimuovere (o saltare) un elemento

In alcuni esercizi, il contatore è utile per capire quando svolgere una particolare richiesta invece che sommare o moltiplicare. Un esempio perfetto di ciò è l'esercizio di Marzo 2024.

```
Implementare in Prolog il predicato remove(L1, K, L2) che vero quando
    la lista L2 si ottiene dalla lista
L1 rimuovendo il K-esimo elemento (contando da 0). Esempio:
?- remove([a, b, c, d, e], 1, L2).
L2 = [a, c, d, e].
```

A primo impatto può sembrare un esercizio molto difficile, che magari richiede l'uso di predicati particolari, in realtà basta capire quando poter "saltare" l'elemento durante lo scorrimento della lista, vediamo come in questo esempio:

```
remove(A, B, C):- remove(A, B, C, 0).
remove([H|T], B, [H|X], D):- D\=B, Y is D+1, remove(T, B, X, Y).
remove([H|T], B, X, B):- Y is B+1, remove(T, B, X, Y).
remove([], _, [], _).
```

Come potete vedere, anche se la lunghezza sintattica è leggermente superiore, la logica dietro è molto semplice, ovvero:

- Creiamo un contatore.
- Se il contatore coincide con il numero dello slot, saltiamo l'elemento nella nuova lista.
- Altrimenti, procediamo normalmente.

```
remove(A, B, C):- remove(A, B, C, 0).
```

Prima di tutto, creiamo il nostro contatore inizializzato a 0 attraverso l'overloading.

```
remove([H|T], B, [H|X], D):- D\=B, Y is D+1, remove(T, B, X, Y).
```

Ora svolgiamo il caso in cui dobbiamo aggiungere gli elementi nella nuova lista, ovvero quando il nostro contatore e B non coincidono, questo è possibile farlo usando la stessa tecnica del primo esercizio per riempire una nuova lista già dichiarata nella query. Ovviamente incrementiamo il nostro contatore di uno con la medesima tecnica che abbiamo visto nell'esercizio precedente.

```
remove([H|T], B, X, B):- Y is B+1, remove(T, B, X, Y).
```

Nel caso in cui sono uguali, stavolta il riempimento della lista nella head non deve avvenire, quindi di limitiamo esclusivamente ad incrementare il contatore, scorrendo comunque la prima lista.

```
remove([], _, [], _).
```

Nel caso finale non servono operazioni particolari.